

Київський національний університет ім. Тараса Шевченка

Кафедра мережевих та інтернет технологій

Лабораторна робота № 4

Дисципліна: Системне програмування

Тема: Асинхронна модель програмування.

Виконав: Студент групи МІТ-41

Полюхович Андрій

Завдання:

1. Розробити програму за шаблоном Windows Form Application, у якій запускається на виконання асинхронна задача (кнопка Start) виконання довготривалої операції. Виконання асинхронної задачі організувати таким чином, щоб вона виконувалася в окремому потоці й елементи керування форми реагували в цей час на дії користувача.
2. Удосконалити розроблену програму таким чином, щоб асинхронна операція повертала результат обчислень та відображала його на елементі керування форми.
3. Доповнити функціонал розробленої програми можливістю відображати хід виконання операції (відсоток у текстовій та графічній формах) за допомогою відповідних елементів керування форми.
4. Розширити функціонал розробленої програми можливістю дочасно перервати виконання довготривалої операції (кнопка Cancel).

Виконання:

файл **Form1.cs**:

```
using System;
using System.Drawing;
using System.Runtime.InteropServices;
using System.Threading;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace Lab
{
    public partial class Form1 : Form
    {
        // [ЗАВДАННЯ 4]: Об'єкт для керування скасуванням
        асинхронної задачі
        private CancellationTokenSource cts;

        // Змінна, яка буде контролювати швидкість роботи (затримку
        в мілісекундах)
        private int delayTime = 50;
```

```

        // Змінна кольору прогрес бару
        [DllImport("user32.dll", CharSet = CharSet.Auto,
SetLastError = false)]
        static extern IntPtr SendMessage(IntPtr hWnd, uint Msg,
IntPtr w, IntPtr l);
        const int PBM_SETSTATE = 1040;
        // -----

        public Form1()
        {
            InitializeComponent();

            this.Text = "Advanced Async Calculator";
            this.BackColor = Color.WhiteSmoke;

            // [ЗАВДАННЯ 1]: Радіокнопки потрібні для демонстрації
            того, що головний потік
            // (інтерфейс) не блокується під час виконання
            довготривалої задачі.
            radioButton1.Text = "Fast (10 ms)";
            radioButton2.Text = "Normal (50 ms)";
            radioButton3.Text = "Slow (200 ms)";
            radioButton2.Checked = true;

            buttonCancel.Enabled = false;
        }

        // [ЗАВДАННЯ 1]: Запуск асинхронної задачі. Метод позначено
        як 'async',
        // що дозволяє використовувати 'await' і не блокувати
        інтерфейс.
        private async void buttonStart_Click(object sender,
EventArgs e)
        {
            buttonStart.Enabled = false;

            buttonCancel.Enabled = true;

            SendMessage(progressBar1.Handle, PBM_SETSTATE,
(IntPtr)1, IntPtr.Zero);

            labelResult.Text = "Result: calculating...";
            labelResult.ForeColor = Color.Black;
            progressBar1.Value = 0;

            // [ЗАВДАННЯ 3]: Інформування про хід виконання.
            // Створюємо об'єкт IProgress, який оновлює відсотки у
            тексті та сам прогрес-бар.
            IProgress<int> onChangeProgress = new Progress<int>((i)
=>
            {
                labelProgress.Text = i.ToString() + "%";
                progressBar1.Value = i;
            });

```

```

        // [ЗАВДАННЯ 4]: Ініціалізація токена скасування перед
        початком роботи.
        cts = new CancellationTokenSource();

        try
        {
            // [ЗАВДАННЯ 1] та [ЗАВДАННЯ 2]:
            // Виклик довготривалої операції через 'await' (не
            блокує форму).
            // Отримання результату обчислень (result) після
            завершення задачі.
            int result = await ProcessAsync(100,
            onChangeProgress, cts.Token);

            // [ЗАВДАННЯ 2]: Відображення отриманого результату
            на елементі керування форми.
            if (!cts.IsCancellationRequested)
            {
                labelResult.Text = "Result: " +
result.ToString();
                labelResult.ForeColor = Color.DarkGreen;
            }
        }
        finally
        {
            buttonStart.Enabled = true;
            buttonCancel.Enabled = false;
        }
    }

    // [ЗАВДАННЯ 4]: Дочасне переривання довготривалої операції
    private void buttonCancel_Click(object sender, EventArgs e)
    {
        if (cts != null && !cts.IsCancellationRequested)
        {
            // Переводимо токен у стан "скасований"
            cts.Cancel();

            SendMessage(progressBar1.Handle, PBM_SETSTATE,
(IntPtr)2, IntPtr.Zero);

            labelResult.Text = "Result: CANCELED ";
            labelResult.ForeColor = Color.Red;
        }
    }

    // --- ОБРОБНИКИ РАДІОКНОПОК (Демонстрація чуйності UI для
    Завдання 1) ---
    private void radioButton1_CheckedChanged(object sender,
    EventArgs e)
    {
        if (radioButton1.Checked) delayTime = 10;
    }

```

```

        private void radioButton2_CheckedChanged(object sender,
EventArgs e)
        {
            if (radioButton2.Checked) delayTime = 50;
        }

        private void radioButton3_CheckedChanged(object sender,
EventArgs e)
        {
            if (radioButton3.Checked) delayTime = 200;
        }

        // --- САМА АСИНХРОННА ОПЕРАЦІЯ ---
        // Метод повертає Task<int>, що відповідає вимогам Завдання
2
        private Task<int> ProcessAsync(int count, IProgress<int>
changeProgressBar, CancellationToken cancellationToken)
        {
            // [ЗАВДАННЯ 1]: Запускаємо код в окремому фоновому
потоці за допомогою Task.Run
            return Task.Run(() =>
            {
                int sum = 0;

                for (int i = 1; i <= count; i++)
                {
                    // [ЗАВДАННЯ 4]: Перевірка того, чи має бути
скасована операція на кожній ітерації
                    if (cancellationToken.IsCancellationRequested)
                    {
                        return sum; // Перериваємо цикл і повертаємо
те, що встигли порахувати
                    }

                    sum += i; // Імітація обчислень (Завдання 2)

                    // [ЗАВДАННЯ 3]: Передаємо поточний відсоток (i)
назад на головну форму
                    changeProgressBar.Report(i);

                    // Штучна затримка, що імітує "довготривалу
операцію"
                    // Змінюється прямо під час роботи через
радіокнопки!
                    Thread.Sleep(delayTime);
                }

                // [ЗАВДАННЯ 2]: Повертаємо фінальний результат
                return sum;
            });
        }
    }
}

```

Результат роботи програми :

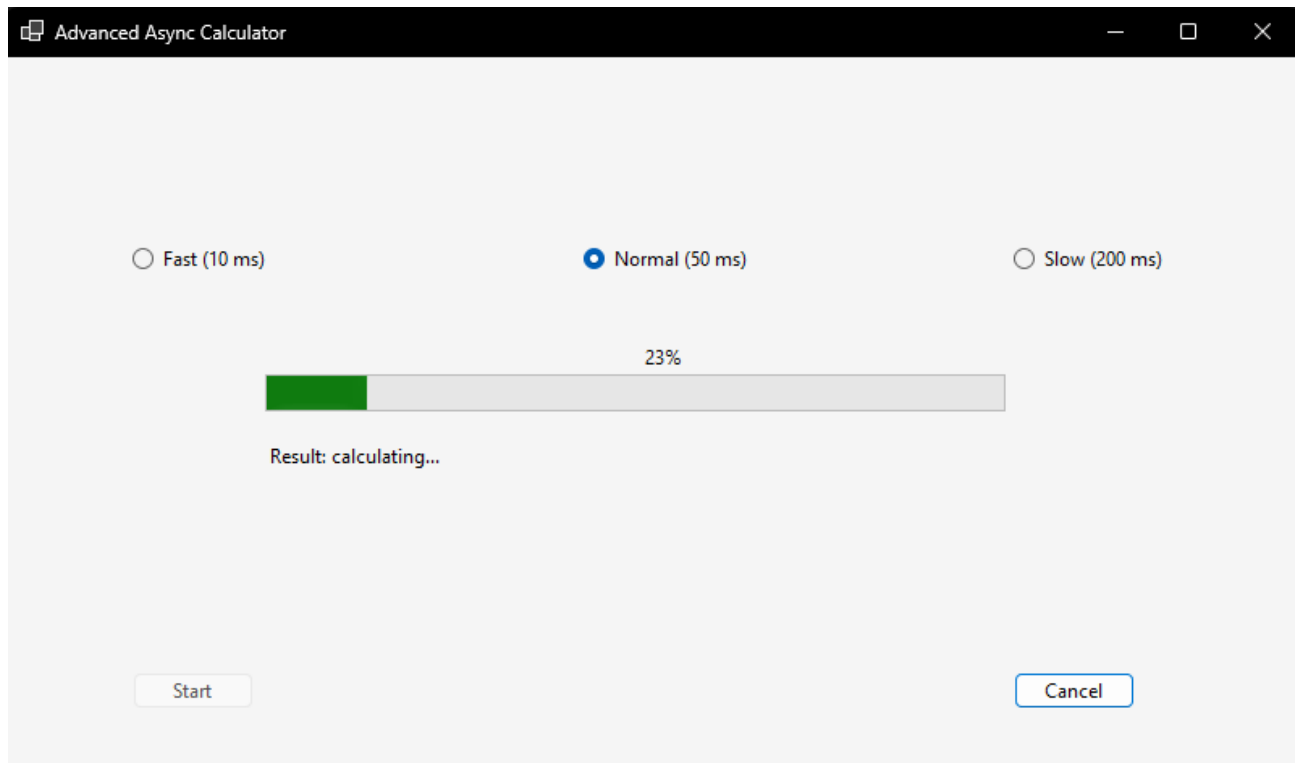


Рисунок 4.1 – програма в процесі роботи.

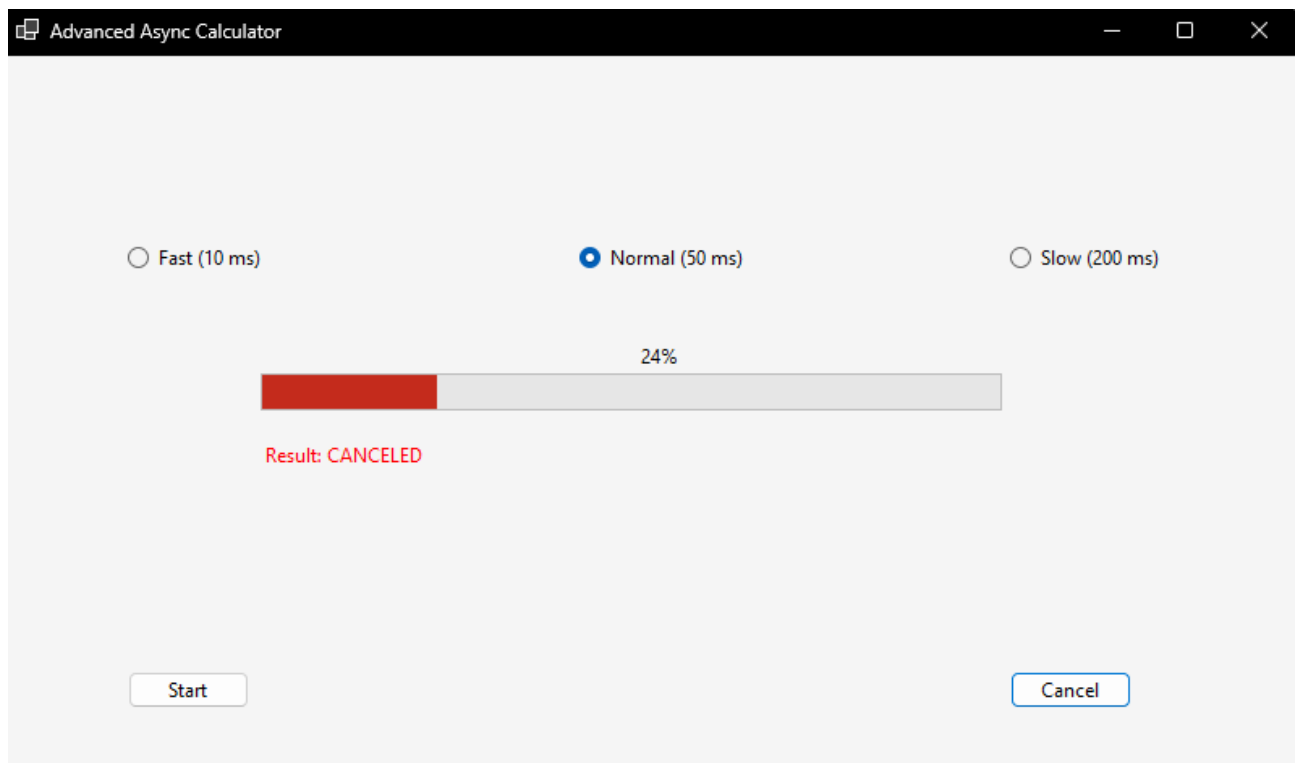


Рисунок 4.2 - програма після натискання кнопки "Cancel"

Опис коду :

Розроблена програма реалізує всі вимоги до асинхронної моделі в середовищі Windows Forms:

Неблокуючий інтерфейс (Завдання 1): Обробник події `buttonStart_Click` позначено ключовим словом `async`. Довготривала операція (`ProcessAsync`) виконується у фоновому потоці за допомогою `Task.Run()`. Оператор `await` дозволяє дочекатися завершення задачі без блокування головного потоку інтерфейсу. Це підтверджується тим, що елементи керування форми (зокрема, перемикачі `RadioButton`) миттєво реагують на дії користувача безпосередньо під час обчислень.

Повернення результату (Завдання 2): Асинхронний метод `ProcessAsync` повертає тип `Task<int>`, що є обіцянкою повернути числове значення після завершення операції. Після виконання циклу результат (сума) передається у головний потік і виводиться на компонент `Label`.

Відображення прогресу (Завдання 3): Для інформування про хід роботи використано інтерфейс `IProgress<int>`. На кожній ітерації фонового циклу викликається метод `Report()`, який передає поточний відсоток виконання на форму. Ці дані динамічно оновлюють текстове поле та графічний елемент `ProgressBar`.

Дочасне переривання (Завдання 4): Скасування операції реалізовано за допомогою `CancellationTokenSource`. При натисканні на кнопку "Cancel" викликається метод `cts.Cancel()`, який переводить токен у скасований стан. Фоновий цикл постійно перевіряє властивість `IsCancellationRequested`, і при отриманні сигналу достроково завершує обчислення.

Висновок

У ході виконання лабораторної роботи було на практиці закріплено принципи асинхронного програмування у C#. Доведено, що використання механізму `async/await` є ефективним способом оптимізації програм із графічним інтерфейсом. Було успішно реалізовано довготривалу задачу, яка виконується у фоні, динамічно передає результати та прогрес на головну форму, дозволяє взаємодіяти з елементами керування в процесі роботи та підтримує безпечне примусове скасування.